

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: EDISON EDUARDO GARCIA VASQUEZ

PARALELO: A

SEMESTRE: 5TO SEMESTRE

ACTIVIDAD: ACTIVIDAD #1: ESTUDIO DE CASO – SISTEMA DE GESTIÓN DE INCIDENTES (HELP DESK)

ENLACE DE GITHUB: [Comparing feature/documentoseg...feature/Documento · Eduardogv98/sistemaseegv](#)

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad _____

Actividad # 1: Estudio de Caso – Sistema de Gestión de Incidentes (Help Desk)

El objetivo de esta actividad es diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos (Help Desk) orientado a la administración de infraestructura de servidores, aplicando patrones de diseño de software y metodologías de trabajo profesionales como Gitflow. A través de este desarrollo, se busca fortalecer las capacidades de análisis y abstracción, promoviendo buenas prácticas en la creación de soluciones escalables, reutilizables, mantenibles y eficientes dentro de entornos de desarrollo colaborativo.

DIAGRAMA DE CASOS DE USOS

1. Elementos del Diagrama de Casos de Uso

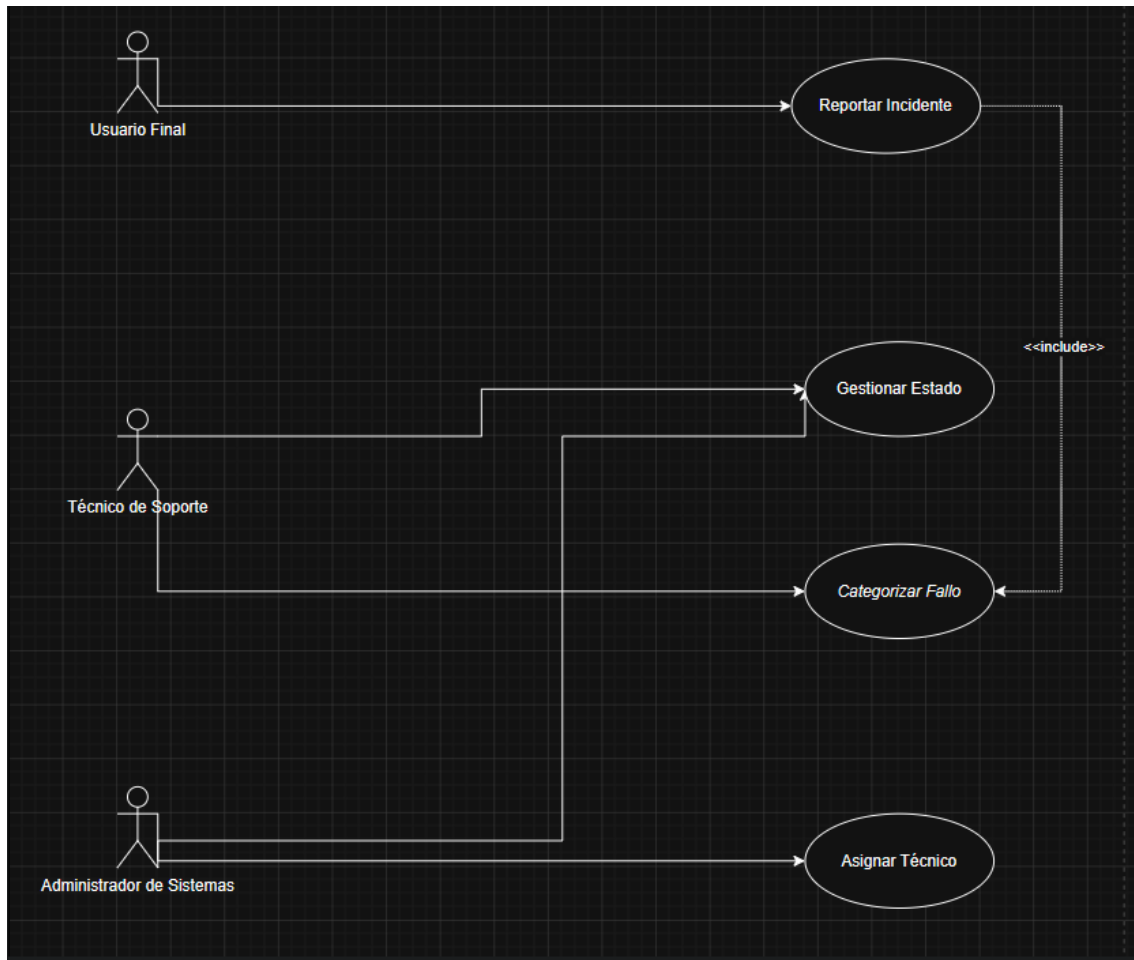
Basado en los requerimientos del documento:

Actores Principalmente Identificados

- Usuario Final: Persona que experimenta el problema técnico y reporta el incidente.
- Técnico de Soporte: Responsable de recibir, atender y resolver los fallos asignados.
- Administrador de Sistemas: Supervisa la plataforma, gestiona usuarios y configuraciones globales.

Casos de Uso (Funcionalidades)

- Reportar Incidente (Crear Ticket): Acción de ingresar un nuevo fallo al sistema.
- Categorizar Fallo: Clasificar si es de Hardware, Red o Software.
- Asignar Técnico: Vincular un especialista al incidente reportado.
- Gestionar Estado de Resolución: Actualizar el avance (ej. "En proceso", "Resuelto").
- Notificar Incidente: Sistema automático que avisa al técnico (relacionado con el patrón *Observer* mencionado en la guía).

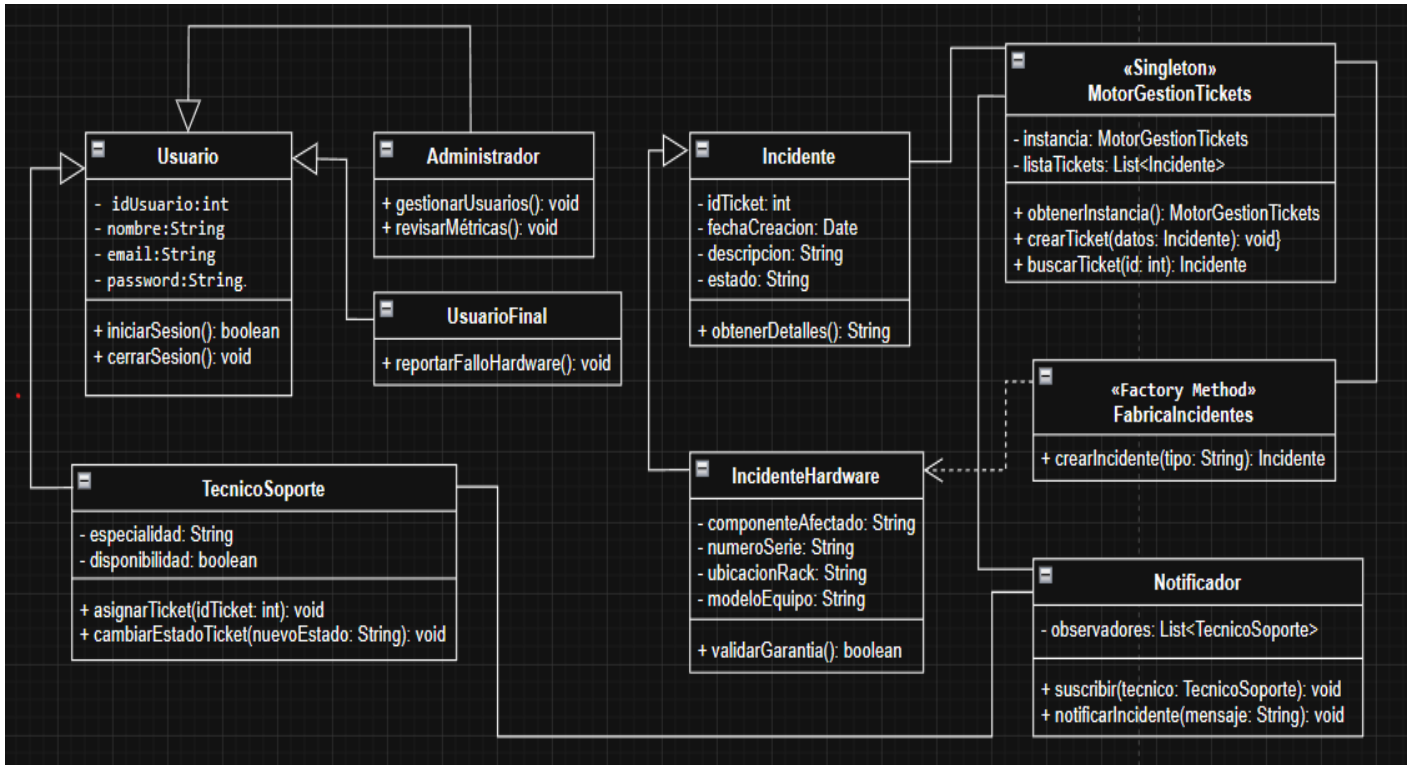


2. Justificación del Diagrama de Clases UML

El diseño del diagrama de clases responde a la necesidad de estructurar una solución bajo el Paradigma Orientado a Objetos y los principios SOLID.

- **Abstracción y Encapsulamiento:** Permite definir la estructura interna de las entidades (como Usuario, Incidente e IncidenteHardware), protegiendo los datos mediante modificadores de acceso (privados - y públicos +) para garantizar la integridad de la información del sistema.
- **Aplicación de Herencia (Especialización):** Se utiliza para optimizar la reutilización de código. Por ejemplo, al hacer que IncidenteHardware herede de Incidente, se evita duplicar atributos comunes como el ID o la fecha, permitiendo que la clase hija se enfoque únicamente en detalles específicos del hardware como el rack o el número de serie.
- **Definición de Relaciones y Dependencias:** Establece cómo se comunican los objetos entre sí (asociaciones y agregaciones). Esto es crucial para asegurar la mantenibilidad y escalabilidad del sistema, permitiendo que el software crezca sin volverse complejo o difícil de reparar.

- Base para el Desarrollo y Control de Versiones: El diagrama sirve como el plano arquitectónico que guía la implementación en el repositorio de GitHub, asegurando que todos los desarrolladores sigan la misma estructura lógica definida en el flujo de Gitflow.



1. Justificación del Diagrama de Casos de Uso

El diseño de este diagrama es fundamental para establecer el **alcance funcional** del sistema Help Desk.

- **Identificación de Fronteras:** Permite delimitar qué procesos pertenecen al sistema de gestión de incidentes y qué acciones son externas, estableciendo el "System Boundary" dentro de la infraestructura del Data Center.
- **Validación de Requerimientos:** Asegura que se han cubierto los requerimientos funcionales solicitados: creación de tickets, categorización de fallos de hardware y asignación de técnicos.
- **Modelado de Interacciones:** Facilita la comprensión de cómo los distintos actores (Usuario, Técnico, Administrador) interactúan con el software, definiendo sus roles y permisos de forma visual antes de pasar a la codificación.

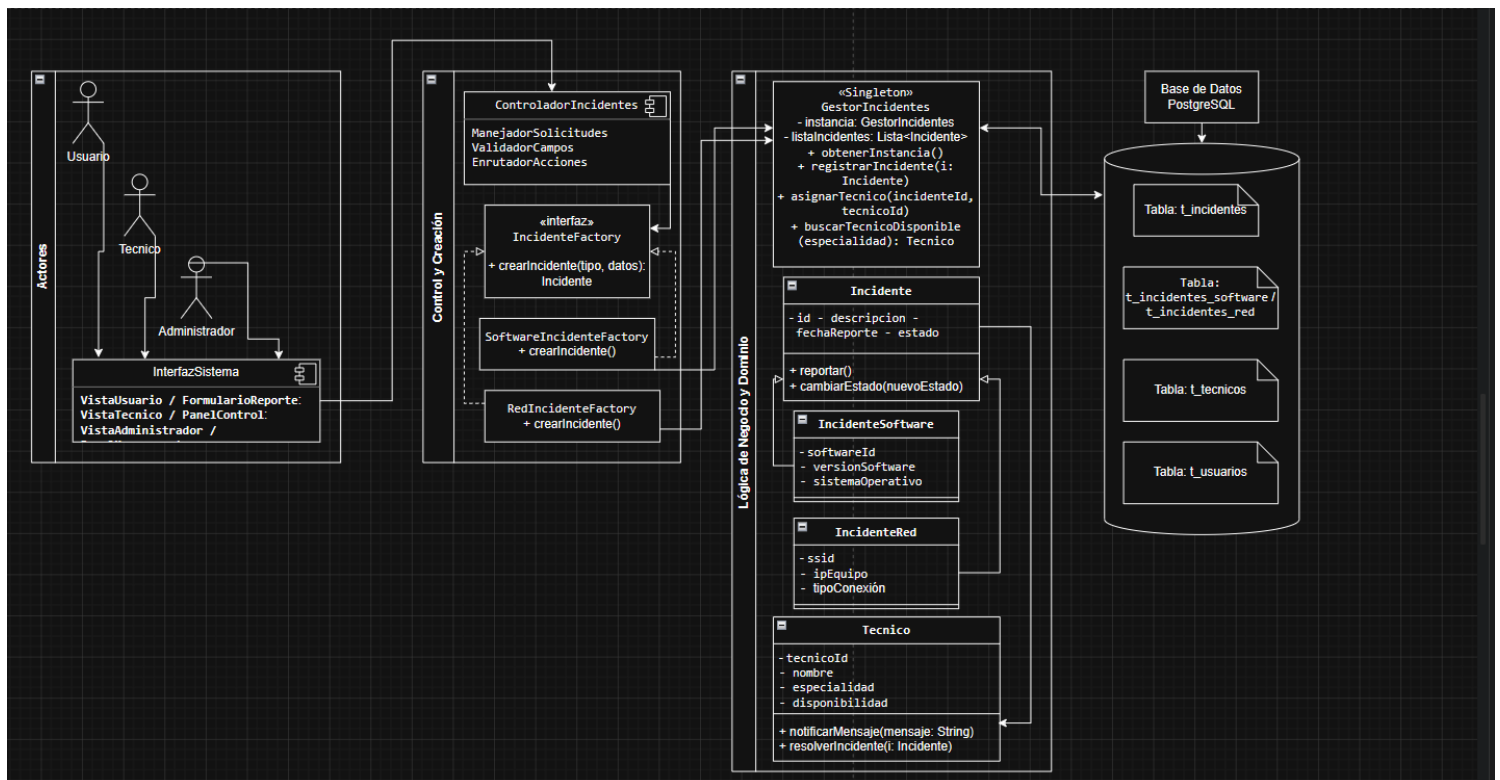
2. Justificación del Diagrama de Clases UML

El diseño del diagrama de clases responde a la necesidad de estructurar una solución bajo el **Paradigma Orientado a Objetos** y los principios **SOLID**.

- **Abstracción y Encapsulamiento:** Permite definir la estructura interna de las entidades (como Usuario, Incidente e IncidenteHardware), protegiendo los

datos mediante modificadores de acceso (privados - y públicos +) para garantizar la integridad de la información del sistema.

- **Aplicación de Herencia (Especialización):** Se utiliza para optimizar la reutilización de código. Por ejemplo, al hacer que IncidenteHardware herede de Incidente, se evita duplicar atributos comunes como el ID o la fecha, permitiendo que la clase hija se enfoque únicamente en detalles específicos del hardware como el rack o el número de serie.
- **Definición de Relaciones y Dependencias:** Establece cómo se comunican los objetos entre sí (asociaciones y agregaciones). Esto es crucial para asegurar la **mantenibilidad y escalabilidad** del sistema, permitiendo que el software crezca sin volverse complejo o difícil de reparar.
- **Base para el Desarrollo y Control de Versiones:** El diagrama sirve como el plano arquitectónico que guía la implementación en el repositorio de GitHub, asegurando que todos los desarrolladores sigan la misma estructura lógica definida en el flujo de Gitflow.



3. Justificación del Diagrama de Arquitectura de Sistemas

"El diagrama de sistemas adjunto representa la arquitectura de software del Sistema de Gestión de Incidentes (Help Desk) mediante un enfoque Multicapa estructurado bajo el patrón MVC (Modelo-Vista-Controlador).

El sistema segmenta claramente las responsabilidades: los agentes externos (Usuario, Técnico y Administrador) interactúan exclusivamente con la capa de Presentación (Vista). Las peticiones son capturadas por la capa de Control, la cual delega la creación de instancias complejas a las fábricas específicas de software y redes (IncidenteFactory), evitando el acoplamiento directo. Una vez creadas las entidades del Modelo (Incidente y Técnico), estas son administradas en memoria por el componente centralizado GestorIncidentes, el cual interactúa finalmente con la capa de Persistencia para asegurar el almacenamiento seguro y relacional de la información en la base de datos."

¿Por qué se diseñó el sistema de esta manera?

Garantía de Escalabilidad y Mantenibilidad: Al utilizar una arquitectura en capas, cualquier cambio en la interfaz gráfica (Capa de Presentación) no afectará las reglas de negocio ni la estructura de la base de datos. Esto permite actualizaciones del sistema sin riesgo de corromper el núcleo del software.

Desacoplamiento Mediante Patrones Implícitos: La inclusión de las fábricas (Factory Method) se justifica porque el sistema maneja diferentes naturalezas de incidentes (Software y Redes). Agregar un nuevo tipo de incidente en el futuro (por ejemplo, Hardware) solo requerirá crear una nueva fábrica, cumpliendo con el principio de Abierto/Cerrado (SOLID).

Integridad de los Datos (Singleton): La centralización a través de un único GestorIncidentes asegura que todos los actores (Administradores y Técnicos) visualicen el mismo estado en tiempo real, previniendo problemas de concurrencia o duplicidad de reportes.

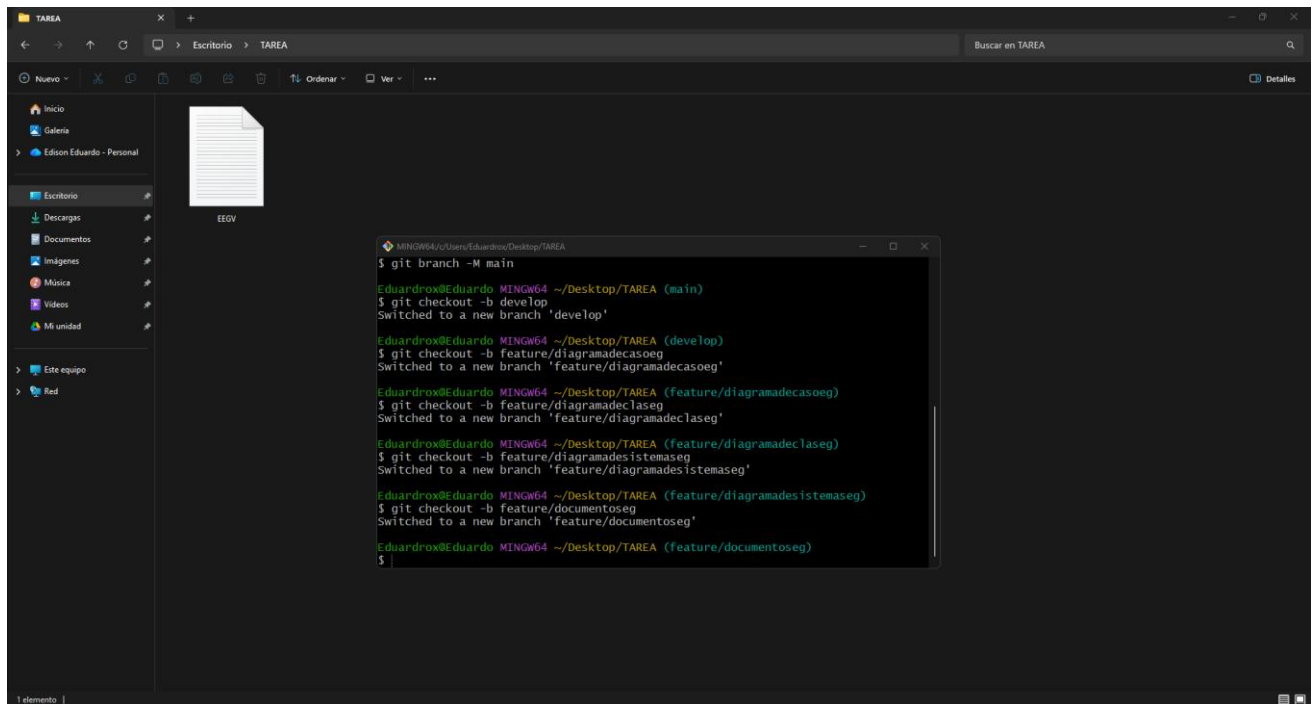
Facilidad para el Trabajo en Equipo (Gitflow): Esta clara separación de componentes permite que un desarrollador trabaje en el Front-end (Vistas) mientras otro modifica las



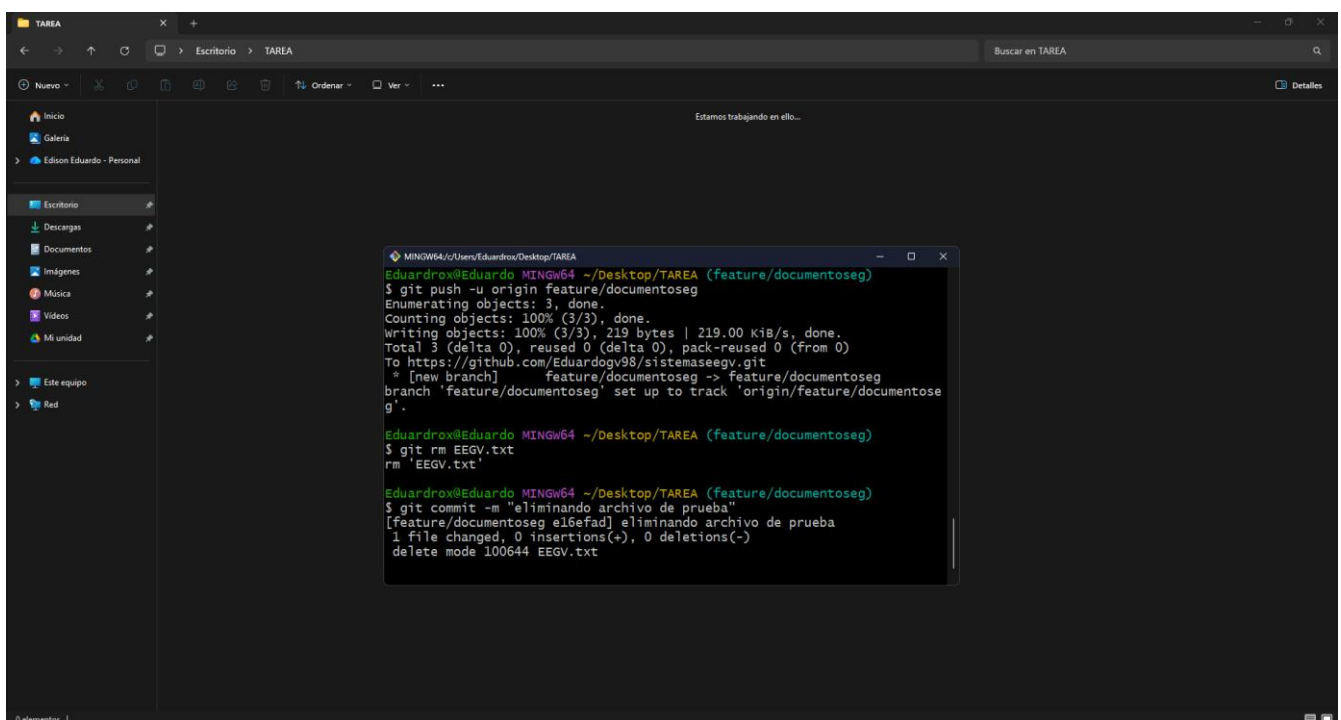
consultas en el Back-end (Persistencia) en ramas separadas, minimizando los conflictos de código (merge conflicts).

Evidencias

(Código de la ejecución de los ejercicios, enlace drive donde se encuentren los mismos y Capturas de pantalla claras y relevantes, donde evidencie el proceso realizado).



```
MINGW64~/Users/Eduardo/Desktop/TAREA
$ git branch -M main
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (main)
$ git checkout -b develop
Switched to a new branch 'develop'
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (develop)
$ git checkout -b feature/diagramadecasoeg
Switched to a new branch 'feature/diagramadecasoeg'
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/diagramadecasoeg)
$ git checkout -b feature/diagramadecaseg
Switched to a new branch 'feature/diagramadecaseg'
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/diagramadecaseg)
$ git checkout -b feature/diagramadesistemaseg
Switched to a new branch 'feature/diagramadesistemaseg'
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/diagramadesistemaseg)
$ git checkout -b feature/documentoseg
Switched to a new branch 'feature/documentoseg'
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/documentoseg)
$
```



```
MINGW64~/Users/Eduardo/Desktop/TAREA
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/documentoseg)
$ git push -u origin feature/documentoseg
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 219 bytes | 219.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Eduardogv98/sistemaseegv.git
 * [new branch] feature/documentoseg -> feature/documentoseg
branch 'feature/documentoseg' set up to track 'origin/feature/documentoseg'.
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/documentoseg)
$ git rm EEGV.txt
rm 'EEGV.txt'
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/documentoseg)
$ git commit -m "eliminando archivo de prueba"
[feature/documentoseg e16efad] eliminando archivo de prueba
1 file changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 EEGV.txt
```



```
MINGW64:/c/Users/Eduardrox/Desktop/TAREA
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/documentoseg)
$ git rm EEGV.txt
rm 'EEGV.txt'

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/documentoseg)
$ git commit -m "eliminando archivo de prueba"
[feature/documentoseg e16efad] eliminando archivo de prueba
1 file changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 EEGV.txt

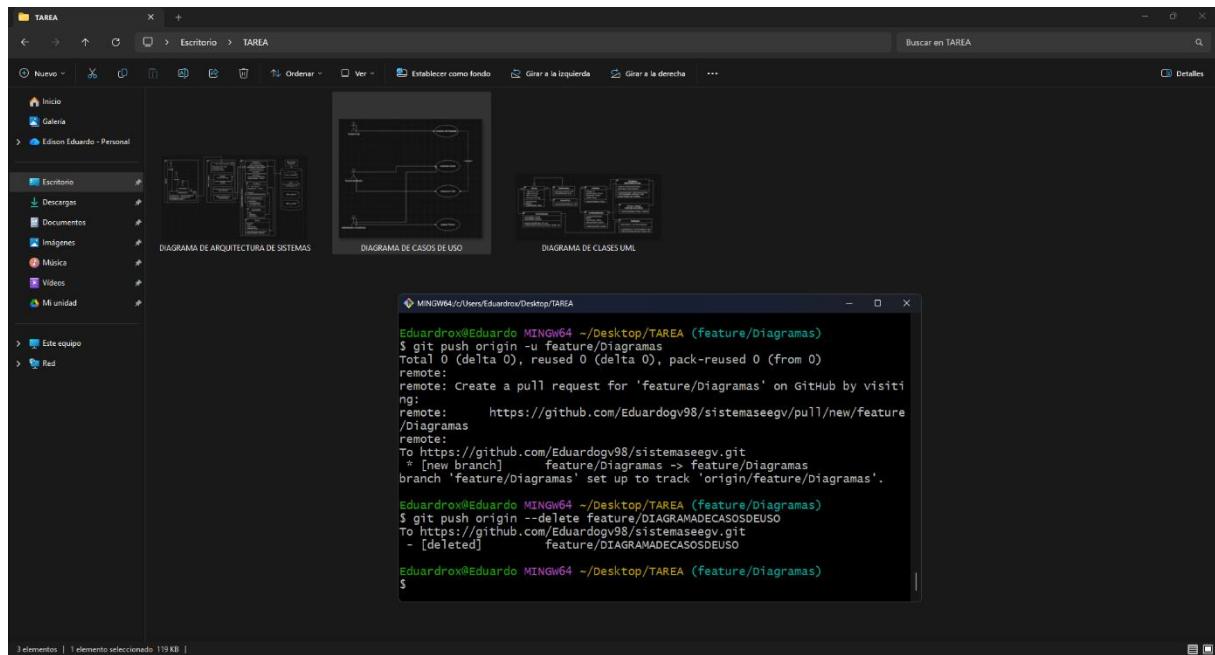
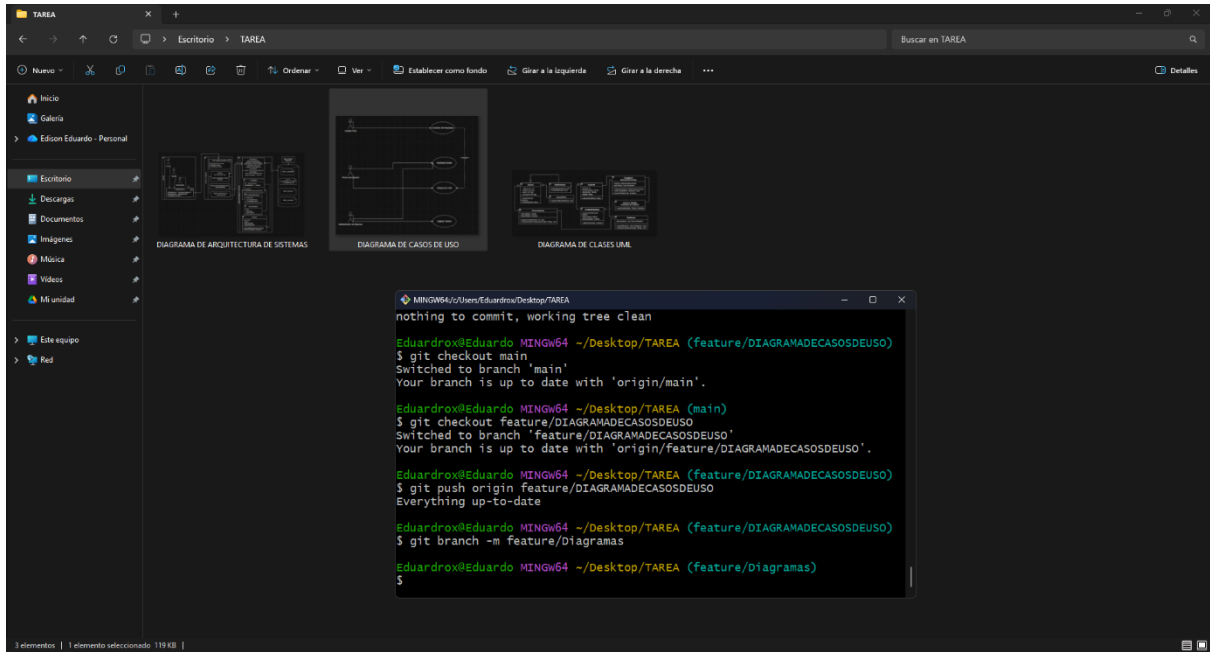
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/documentoseg)
$ git push
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (2/2), 207 bytes | 207.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Eduardogv98/sistemaseegv.git
8165b7b..e16efad feature/documentoseg -> feature/documentoseg

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/documentoseg)
$
```

```
MINGW64:/c/Users/Eduardrox/Desktop/TAREA
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (develop)
$ git checkout -b feature/DIAGRAMADECASOSDEUSO
Switched to a new branch 'feature/DIAGRAMADECASOSDEUSO'

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/DIAGRAMADECASOSDEUSO)
$ git push -u origin feature/DIAGRAMADECASOSDEUSO
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/DIAGRAMADECASOSDEUSO' on GitHub by visiting:
remote: https://github.com/Eduardogv98/sistemaseegv/pull/new/feature/DIAGRAMADECASOSDEUSO
remote:
To https://github.com/Eduardogv98/sistemaseegv.git
* [new branch] feature/DIAGRAMADECASOSDEUSO -> feature/DIAGRAMADECASOSDEUSO
branch 'feature/DIAGRAMADECASOSDEUSO' set up to track 'origin/feature/DIAGRAMADECASOSDEUSO'.

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/DIAGRAMADECASOSDEUSO)
$ |
```





```
MINGW64/c/Users/Eduardox/Desktop/TAREA
- [deleted] feature/DIAGRAMADECASOSDEUSO

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/Diagramas)
$ git checkout develop
Switched to branch 'develop'
Your branch is up to date with 'origin/develop'.

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (develop)
$ git rm *.png
rm 'DIAGRAMA DE ARQUITECTURA DE SISTEMAS.png'
rm 'DIAGRAMA DE CASOS DE USO.png'
rm 'DIAGRAMA DE CLASES UML.png'

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (develop)
$ git commit -m "chore: eliminar archivos png de la rama develop"
[develop f41e87a] chore: eliminar archivos png de la rama develop
3 files changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 DIAGRAMA DE ARQUITECTURA DE SISTEMAS.png
delete mode 100644 DIAGRAMA DE CASOS DE USO.png
delete mode 100644 DIAGRAMA DE CLASES UML.png

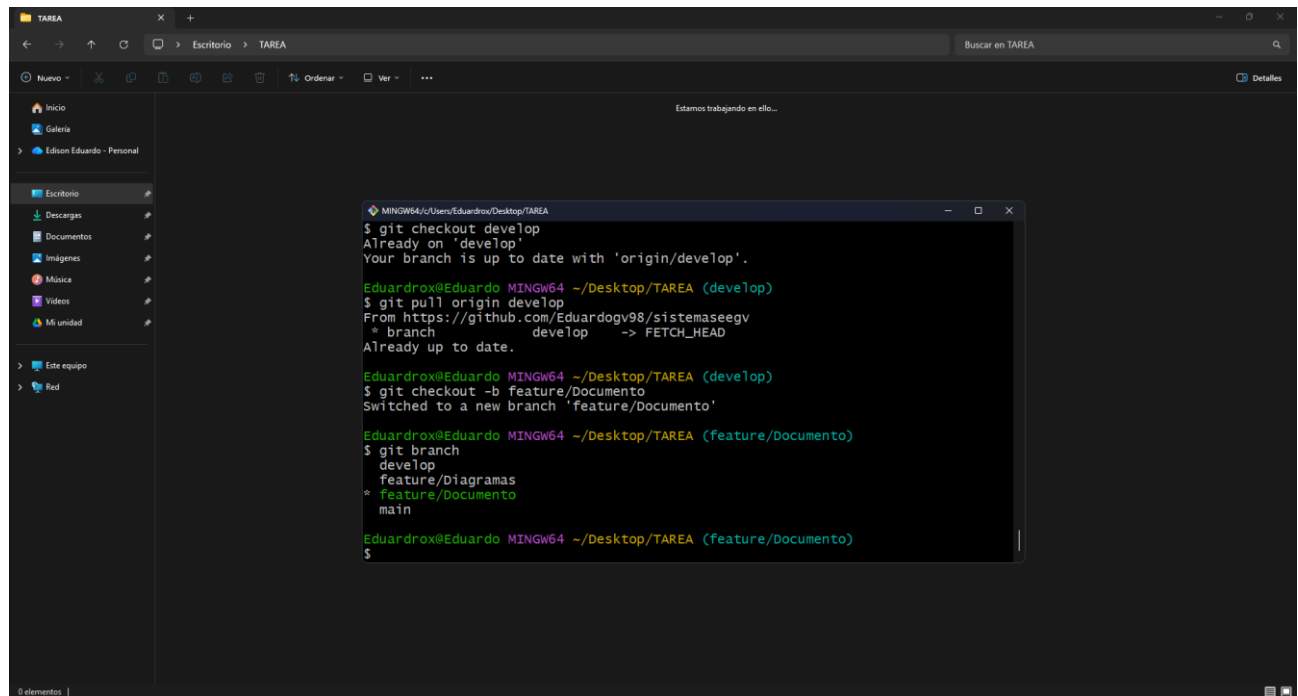
Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (develop)
$
```

```
MINGW64/c/Users/Eduardox/Desktop/TAREA
rm 'DIAGRAMA DE CLASES UML.png'

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (develop)
$ git commit -m "chore: eliminar archivos png de la rama develop"
[develop f41e87a] chore: eliminar archivos png de la rama develop
3 files changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 DIAGRAMA DE ARQUITECTURA DE SISTEMAS.png
delete mode 100644 DIAGRAMA DE CASOS DE USO.png
delete mode 100644 DIAGRAMA DE CLASES UML.png

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (develop)
$ git push origin develop
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 16 threads
Compressing objects: 100% (1/1), done.
Writing objects: 100% (2/2), 220 bytes | 220.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Eduardogv98/sistemaseegv.git
 36025fb..f41e87a develop -> develop

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (develop)
$
```



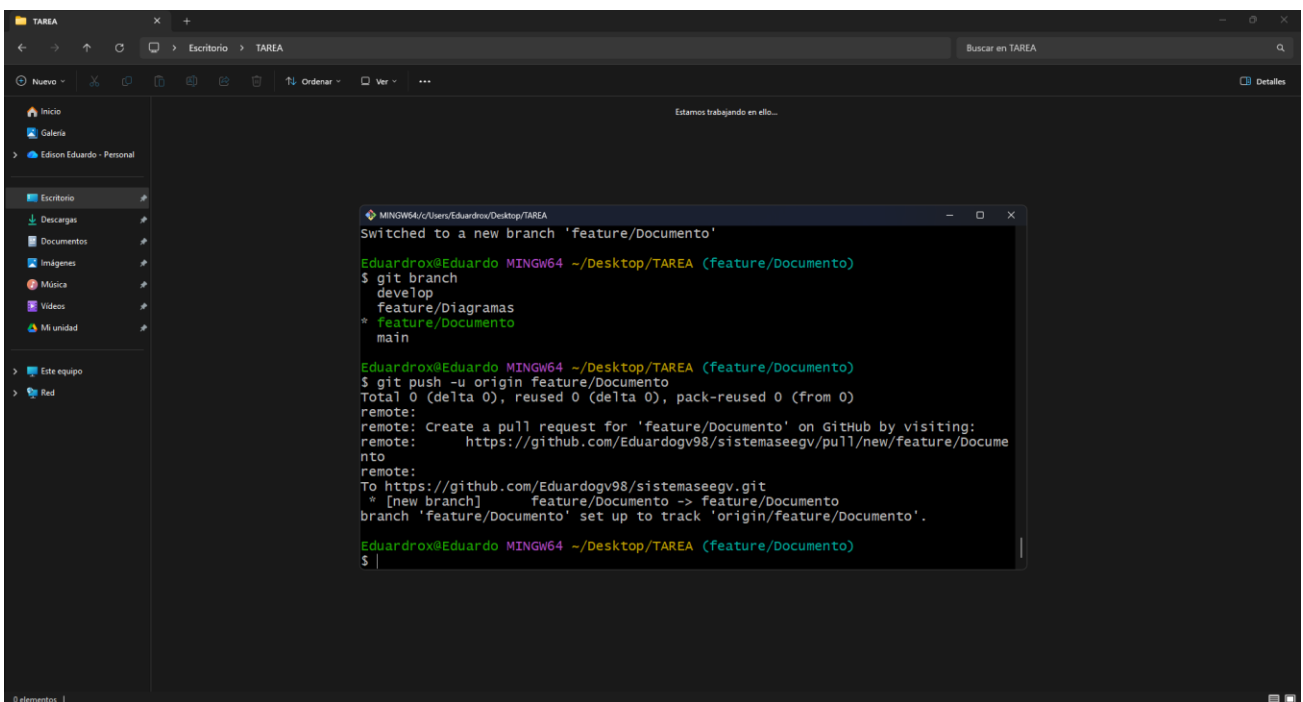
```
MINGW64/c/Users/Eduardox/Desktop/TAREA
$ git checkout develop
Already on 'develop'
Your branch is up to date with 'origin/develop'.

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (develop)
$ git pull origin develop
From https://github.com/Eduardogv98/sistemaseegv
 * branch      develop    -> FETCH_HEAD
Already up to date.

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (develop)
$ git checkout -b feature/Documento
Switched to a new branch 'feature/Documento'

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/Documento)
$ git branch
  develop
  feature/Diagramas
  * feature/Documento
  main

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/Documento)
$
```



```
MINGW64/c/Users/Eduardox/Desktop/TAREA
Switched to a new branch 'feature/Documento'

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/Documento)
$ git branch
  develop
  feature/Diagramas
  * feature/Documento
  main

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/Documento)
$ git push -u origin feature/Documento
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/Documento' on GitHub by visiting:
remote:   https://github.com/Eduardogv98/sistemaseegv/pull/new/feature/Documento
remote:
To https://github.com/Eduardogv98/sistemaseegv.git
 * [new branch]      feature/Documento -> feature/Documento
branch 'feature/Documento' set up to track 'origin/feature/Documento'.

Eduardrox@Eduardo MINGW64 ~/Desktop/TAREA (feature/Documento)
$
```